1. Javascript intro - syntax, output, comments
2. variables
3. Operators
4. Data Types
5. objects
6. arrays
7. functions
8. loops
9. java script events - onclick, onfocus, onblur, keypress, onmouseover, onmouseout
10. Scope
11. Hoisting
12. this keyword
13. Arrow functions
14. Json
15. Dom
16. BOM
17. JS-Browser
18. validations
19. callbacks, promises, async/Await - react

What is an Operator?
In JavaScript, an operator is a special symbol used to perform operations on operands (values and variables). For example,
2 + 3; // 5

Here + is an operator that performs addition, and 2 and 3 are operands.

List of different operators you will learn in this tutorial.

1. Assignment Operators
2. Arithmetic Operators
3. Comparison Operators
4. Logical Operators
5. Bitwise Operators
6. String Operators
7. Other Operators

Assignment Operators

= Assignment operator a = 7; // 7
+= Addition assignment a += 5; // a = a + 5
-= Subtraction Assignment a -= 2; // a = a - 2
*= Multiplication Assignment a *= 3; // a = a * 3
/= Division Assignment a /= 2; // a = a / 2
%= Remainder Assignment a %= 2; // a = a % 2
= Exponentiation Assignment a **= 2; // a = a2

Arithmetic Operators

+ Addition x + y
- Subtraction x - y
* Multiplication x * y
/ Division x / y
% Remainder x % y
++ Increment (increments by 1) ++x or x++
-- Decrement (decrements by 1) --x or x--
** Exponentiation (Power) x ** y - note

example of Arithmetic operators:

```
let x = 5;
let y = 3;

// addition
console.log('x + y = ', x + y);  // 8

// subtraction
console.log('x - y = ', x - y);  // 2

// multiplication
console.log('x * y = ', x * y);  // 15

// division
console.log('x / y = ', x / y);  // 1.6666666666666667

// remainder
console.log('x % y = ', x % y);   // 2
```

```
// increment
console.log('++x = ', ++x); // x is now 6
console.log('x++ = ', x++); // prints 6 and then increased to 7
console.log('x = ', x);     // 7

// decrement
console.log('--x = ', --x); // x is now 6
console.log('x-- = ', x--); // prints 6 and then decreased to 5
console.log('x = ', x);     // 5

//exponentiation
console.log('x ** y =', x ** y);
```

Comparison Operators

== Equal to: returns true if the operands are equal x == y
!= Not equal to: returns true if the operands are not equal x != y
=== Strict equal to: true if the operands are equal and of the same type x === y
!== Strict not equal to: true if the operands are equal but of different type or not equal at all x !== y
> Greater than: true if left operand is greater than the right operand x > y
>= Greater than or equal to: true if left operand is greater than or equal to the right operand x >= y
< Less than: true if the left operand is less than the right operand x < y
<= Less than or equal to: true if the left operand is less than or equal to the right operand x <= y
Examples of comparison operators

```
// equal operator
console.log(2 == 2); // true
console.log(2 == '2'); // true

// not equal operator
console.log(3 != 2); // true
console.log('hello' != 'Hello'); // true

// strict equal operator
console.log(2 === 2); // true
console.log(2 === '2'); // false

// strict not equal operator
console.log(2 !== '2'); // true
console.log(2 !== 2); // false
```

Logical Operators

&& Logical AND: true if both the operands are true, else returns false x && y
|| Logical OR: true if either of the operands is true; returns false if both are false x || y
! Logical NOT: true if the operand is false and vice-versa. !x

example:

```
const x = 5, y = 3;
(x < 6) && (y < 5); // true
```

Bitwise Operators:

String Operators:

+ is a concatenation operator

example

```
// concatenation operator
console.log('hello' + 'world');
let a = Hi;
a += ' Class;  // a = a + ' Class;
console.log(a);
```

Other Operators

, (comma)
evaluates multiple operands and returns the value of the last operand.

```
let a = (1, 3 , 4); // 4
```

?:
returns value based on the condition

```
 (6 > 3) ? 'success' : 'error'; // "success"
```

delete
deletes an object's property, or an element of an array

```
delete x
```

typeof

returns a string indicating the data type.

```
typeof 3; // "number"
```

void

discards the expression's return value

```
void(x)
```

in

returns true if the specified property is in the object

```
Prop in object
```

InstanceOf

returns true if the specified object is of of the specified object type

```
object instanceof object_type
```

Data Types:

Data Types	Description	Example
`String`	represents textual data	`'hello'`, `"hello world!"` etc
`Number`	an integer or a floating-point number	`3`, `3.234`, `3e-2` etc.
`BigInt`	an integer with arbitrary precision	`900719925124740999n` , `1n` etc.
`Boolean`	Any of two values: true or false	`true` and `false`
`undefined`	a data type whose variable is not initialized	`let a;var a;`
`null`	denotes a `null` value	`let a = null;`
`Symbol`	data type whose instances are unique and immutable	`let value = Symbol('hello');`
`Object`	key-value pairs of collection of data	`let student = { };`

Here, all data types except `Object` are primitive data types, whereas `Object` is non-primitive.The object data type can contain:

1. An object
2. An array
3. A date

JavaScript String

String is used to store text. In JavaScript, strings are surrounded by quotes:

- Single quotes: 'Hello'
- Double quotes: "Hello"
- Backticks: `Hello`

```
//strings example
const name = 'ram';
const name1 = "hari";
const result = `The names are ${name} and ${name1}`;
```

Single quotes and double quotes are practically the same and you can use either of them.

Backticks are generally used when you need to include variables or expressions into a string. This is done by wrapping variables or expressions with ${variable or expression} as shown above.

JavaScript Number

Number represents integer and floating numbers (decimals and exponentials). For example,

```
const number1 = 3;
const number2 = 3.433;
const number3 = 3e5 // 3 * 10^5
```

A number type can also be +Infinity, -Infinity, and NaN (not a number). For example,

```
const number1 = 3/0;
console.log(number1); // Infinity

const number2 = -3/0;
console.log(number2); // -Infinity

// strings can't be divided by numbers
const number3 = "abc"/3;
console.log(number3);  // NaN
```

JavaScript BigInt

`BigInt` was introduced in the newer version of JavaScript and is not supported by many browsers including Safari.

JavaScript Boolean

This data type represents logical entities. Boolean represents one of two values: true or false. It is easier to think of it as a yes/no switch. For example,

```
const dataChecked = true;
const valueCounted = false;
```

JavaScript undefined

The undefined data type represents a value that is not assigned. If a variable is declared but the value is not assigned, then the value of that variable will be undefined.

```
let name;

console.log(name); // undefined
```

JavaScript null

In JavaScript, `null` is a special value that represents an empty or **unknown value**.

```
const number = null;
```

The code above suggests that the *number* variable is empty.

Note: `null` is not the same as NULL or Null.

JavaScript Object

An `object` is a complex data type that allows us to store collections of data. For example,

```
const student = {
 firstName: 'ram',
  class: 10
};
```

JavaScript typeof

To find the type of a variable, you can use the `typeof` operator. For example,

```
const name = 'ram';

typeof(name); // returns "string"

const number = 4;

typeof(number); //returns "number"

const valueChecked = true;

typeof(valueChecked); //returns "boolean"

const a = null;

typeof(a); // returns "object"
```

Notice that `typeof` returned `"object"` for the `null` type. This is a known issue in JavaScript since its first release.

Objects:

JavaScript object is a non-primitive data-type that allows you to store multiple collections of data.

Note: If you are familiar with other programming languages, JavaScript objects are a bit different. You do not need to create classes in order to create objects.

The syntax to declare an object is:

```
const object_name = {
  key1: value1,
  key2: value2
}
```

You can also define an object in a single line.

You can access the **value** of a property by using its **key**.

```
const person = {
   name: 'John',
   age: 20,
};
// accessing property
console.log(person.name); // John
```

Using bracket notation, you can access the property of object - objectName["propertyName"]

It is a common practice to declare objects with the const keyword.

Arrays

An array is an object that can store multiple values at once.

We can create an array by placing elements inside an array literal `[ ]`, separated by commas. For example,

```
const numbers = [10, 30, 40, 60, 80]
```

Here,

- `numbers` - name of the array
- `[10, 30, 40, 60, 80]` - elements of the array

We can use an array index to access the elements of the array.

- `numbers[0]` - access the first element **1**.
- `numbers[2]` - access the third element **3**.

```
let dailyActivities = ['eat', 'sleep'];

// add an element at the end
```

```
dailyActivities.push('exercise');

console.log(dailyActivities);
```

We can remove an element from any specified index of an array using the `splice()` method.

We can add elements to an array using built-in methods like `push()` and `unshift()`.

- The `push()` method adds an element at the end of the array.

Array Methods

In JavaScript, there are various methods available that make it easier to perform useful operations with arrays. Some commonly used array methods in JavaScript are:

Method	Description
concat()	Joins two or more arrays and returns a result.
indexOf()	Searches an element of an array and returns its position.
find()	Returns the first value of an array element that passes a test.
findIndex()	Returns the first index of an array element that passes a test.
forEach()	Calls a function for each element.
includes()	Checks if an array contains a specified element.
sort()	Sorts the elements alphabetically in strings and in ascending order.
slice()	Selects the part of an array and returns the new array.

splice()	Removes or replaces existing elements and/or adds new elements.

Functions

A JavaScript function is a block of code designed to perform a particular task.

A JavaScript function is executed when "something" invokes it (calls it).

```
// Function to compute the product of p1 and p2
function myFunction(p1, p2) {
  return p1 * p2;
}
```

Loops

For loop

While

do-while

Javascript Events

Scope

Difference Between var, let and const

	Scope	Redeclare	Reassign	Hoisted	Binds this

var	No	Yes	Yes	Yes	Yes
let	Yes	No	Yes	No	No
const	Yes	No	No	No	No

Block Scope

Before ES6 (2015), JavaScript did not have **Block Scope**.

JavaScript had **Global Scope** and **Function Scope**.

ES6 introduced the two new JavaScript keywords: `let` and `const`.

These two keywords provided **Block Scope** in JavaScript:

Global Scope

Variables declared with the `var` always have **Global Scope**.

Variables declared with the `var` keyword can NOT have block scope:

Function Scope

Variables declared within a JavaScript function, become **LOCAL** to the function.

Local variables can only be accessed from within the function.

Hoisting

Hoisting is JavaScript's default behavior of moving **declarations** to the top of the current scope.

Hoisting applies to variable declarations and to function declarations.

Because of this, JavaScript functions can be called before they are declared:

```
myFunction(5);

function myFunction(y) {
  return y * y;
}
```

Functions defined using an expression are not hoisted.

this keyword

In JavaScript, this keyword refers to an object.

Which object depends on how this is being invoked (used or called).

The this keyword refers to different objects depending on how it is used:

1. In an object method, `this` refers to the **object**.
2. Alone, `this` refers to the **global object**.
3. Alone, `this` refers to the **global object**.
4. In a function, in strict mode, `this` is `undefined`.
5. In an event, `this` refers to the **element** that received the event.

Note

this is not a variable. It is a keyword. You cannot change the value of this.

```
const person = {
```

```
  firstName: "John",
  lastName : "Doe",
  id      : 5566,
  fullName : function() {
        return this.firstName + " " + this.lastName;
  }
};
```

Arrow functions

Arrow functions were introduced in ES6.

Arrow functions allow us to write shorter function syntax:

```
let myFunction = (a, b) => a * b;
```

Before Arrow function

```
hello = function() {
  return "Hello World!";
}
```

After Arrow function

```
hello = () => {
  return "Hello World!";
}
```

Json

JSON is a format for storing and transporting data.
JSON is often used when data is sent from a server to a web page.

What is JSON?

- JSON stands for **J**ava**S**cript **O**bject **N**otation
- JSON is a lightweight data interchange format
- JSON is language independent ***
- JSON is "self-describing" and easy to understand

* The JSON syntax is derived from JavaScript object notation syntax, but the JSON format is text only.
Code for reading and generating JSON data can be written in any programming language.

JSON Example

This JSON syntax defines an employees object: an array of 3 employee records (objects):

JSON Example

```
{
"employees":[
  {"firstName":"John", "lastName":"Doe"},
  {"firstName":"Anna", "lastName":"Smith"},
  {"firstName":"Peter", "lastName":"Jones"}
]
}
```

JSON Object :

```
{"firstName":"John", "lastName":"Doe"}
```

JSON Array :

```
"employees":[
  {"firstName":"John", "lastName":"Doe"},
  {"firstName":"Anna", "lastName":"Smith"},
  {"firstName":"Peter", "lastName":"Jones"}
]
```